DevOps Question Bank Monitoring, Observability, SRE & Scale Design

32 real interview questions - de-duplicated, topic-wise, Basic / Advanced / Scenario-Based
Every answer: plain-English idea · technical answer · example · what to say in the interview

By Priyanka Gupta — 25 September 2026

How to use this question bank

Each question has four parts:

In simple words (green box) - a plain-English analogy. Read this first; if you understand it, the technical part becomes easy to remember.

Detailed answer + Example - the real technical answer, with commands / YAML / code you can show.

What to say to the interviewer (blue box) - 2-3 lines to say out loud. Practise these aloud.

Questions that were asked more than once across the lists were merged into one (the other wordings are shown in grey as 'Also asked as'). Language-specific (Python/PHP) and non-DevOps questions were removed. Where a pasted 'model answer' was wrong, the corrected version is given.

Suggested order: all Basic questions first, then Scenario-Based (most interviews are troubleshooting-heavy), then Advanced. Use the PDF bookmarks panel to jump to any question.

Topic	Basic	Advanced	Scenario	Total
Monitoring, Observability, SRE & Scale Design	9	15	8	32

Questions in this topic

Basic **5**

Q1. Which monitoring and logging tools have you worked with (Prometheus, Grafana, ELK, Loki, Datadog)?

Q2. What is observability, and how is it different from monitoring? 5

Q3. What is the difference between logs, metrics and traces? 7

Q4. What is log aggregation, and why do we need it? 7

Q5. What are SLI, SLO, SLA and error budget? 8

Q6. How does Prometheus collect data from applications in Kubernetes? 8

Q7. How do you monitor Kubernetes clusters and applications, and how do you integrate Prometheus and Grafana with Kubernetes? 9

Q8. What is a runbook, and what does on-call ownership mean? 10

Q9. What is FinOps? What are the basics of cloud cost optimization? 11

Advanced **12**

Q10. How do you implement centralized logging for microservices? 12

Q11. Design a highly available logging system for 100+ microservices across 3 regions. 12

Q12. How do you implement and monitor distributed tracing in microservices? 13

Q13. How would you design an observability stack using CloudWatch, Prometheus and Grafana? . . . 14

Q14. How would you monitor HPA scaling decisions in real time and detect if the metrics server is lagging?

Q15. Your monitoring shows constant false alerts. How do you tune alerts and reduce alert fatigue? . . 15

Q16. You are asked to design an alerting system for high availability. What is your approach? 17

Q17. How would you monitor the end-to-end SLA for services involved in a payments pipeline? 17

Q18. How would you design auto-scaling for 50M+ concurrent viewers across multiple Kubernetes clusters without over-provisioning? 19

Q19. During an IPL final a new region needs to spin up instantly. How would you pre-warm nodes and scale workloads with zero cold-start impact? 20

Q20. How would you use Envoy and Istio to route low-latency live streams differently from VOD without service restarts? 20

Q21. How would you configure readiness and liveness probes to catch buffering or lag in stream-processing services before users notice? 21

Q22. How do you build a culture where latency SLOs are enforced like uptime SLAs? 22

Q23. How would you simulate chaos in a streaming pipeline without risking real user impact? 22

Q24. How do you justify infrastructure costs for pre-warmed scaling capacity to executives before a major sports event? 23

Scenario-Based **24**

Q25. "Production is down. Users are impacted. What will you do?" 24

Q26. Monitoring shows increased response time, but infrastructure (CPU, memory, pods) looks healthy. How would you investigate? 25

Q27. It is 2 AM. Users report the app is slow. There are no errors and no logs. What is your first move? 26

Q28. Application metrics are not being scraped by Prometheus. What could cause it, and how do you validate scraping? 28

Q29. Playback failures spike for only 3% of users in the APAC region. CPU, memory and pods look fine. Walk through your triage plan. 29

Q30. Your Kafka ingestion pipeline lags by 2 minutes during a traffic surge. Producers are fine, consumers are idle. What is your debug path? 29

Q31. Sudden tail latency on the Redis-based stream session store during a Champions League match. How do you find and fix the bottleneck? 30

Q32. After an outage, how would you conduct a proper post-incident analysis (RCA / blameless postmortem)? 31

Monitoring, Observability, SRE & Scale Design

Basic

Q1. Which monitoring and logging tools have you worked with (Prometheus, Grafana, ELK, Loki, Datadog)?

Also asked as: What monitoring tools have you used? Have you worked with Prometheus, Grafana or ELK Stack?

In simple words: Monitoring tools are the dashboard and warning lights of a car. Prometheus measures the numbers (speed, fuel), Loki/ELK keeps the diary of what happened, and Grafana is the screen where you look at all of it.

Detailed answer

Group the tools by the job they do, so the answer sounds organized instead of a list of names:

Job	Common tools	What it gives you
Metrics	Prometheus, CloudWatch, Zabbix, Datadog	Numbers over time: CPU, latency, error rate
Logs	Loki, ELK/EFK (Elasticsearch + Fluentd/Fluent Bit + Kibana), CloudWatch Logs	Text events from apps and systems
Traces	Tempo, Jaeger, OpenTelemetry, X-Ray	The path of one request across services
Visualization	Grafana, Kibana	Dashboards and exploration
Alerting	Alertmanager, Grafana Alerting, Zabbix triggers, CloudWatch Alarms	Notifications to Teams/Slack/email/pager

- **Prometheus** pulls (scrapes) metrics from `/metrics` endpoints every 15-30 seconds and stores them as time series.
- **Loki** stores logs cheaply by indexing only labels (namespace, pod, app), not the full text. ELK indexes everything, so it searches faster but costs more.
- **Grafana** can read all of them in one place.
- **Docker containers** (outside Kubernetes) are monitored with **cAdvisor**, which exposes per-container CPU, memory, network and filesystem metrics on `/metrics` for Prometheus to scrape. `docker stats` gives a quick live view. Inside Kubernetes, cAdvisor is already built into the kubelet.

What to say to the interviewer: *I have worked with Prometheus and Alertmanager for Kubernetes metrics and alerts, Zabbix for host, service and SLA monitoring, Loki for centralized logs, and Grafana as the single dashboard layer over all of them. I think of it as metrics for "is something wrong", logs for "what exactly happened" and traces for "where in the request path it happened".*

Q2. What is observability, and how is it different from monitoring?

Also asked as: What does observability mean to you, and how do you achieve it? Logs are not observability - can you debug user pain?

In simple words: Monitoring is a smoke alarm - it tells you there is a fire in the kitchen. Observability is being able to walk in and figure out which pan caught fire and why, even if nobody predicted that exact fire.

Detailed answer

- **Monitoring** = watching known things with pre-defined checks and thresholds. "Is CPU above 90%?" "Is the site returning 200?" It answers questions you already thought of.
- **Observability** = how well you can understand the internal state of a system from the data it produces, so you can **ask new questions without shipping new code**. "Why are only Android users in Mumbai on app version 5.2 seeing slow checkout since 10:05?"
- **Having logs is not observability.** A pile of unlinked logs cannot tell you which users are hurting or where time is spent. Observability means you can go from a user's pain to the exact cause.

What makes a system observable:

- **High-cardinality context** - every signal carries labels or attributes such as tenant, region, app version, endpoint, and (in traces/logs, not metric labels) user or request ID, so you can slice data many ways.
- **Correlation across signals** - metrics -> traces through **exemplars** (a latency point links to a real trace), traces -> logs through a shared `trace_id`, and the same service/namespace labels everywhere.
- **User-perspective SLOs** - measure what users feel with real-user monitoring (RUM) and synthetic checks, not only server CPU. "99% of checkouts complete in under 2s" beats "pods are Running".

How I would get there:

1. **Instrument with OpenTelemetry** - one standard for traces, metrics and logs, with auto-instrumentation where possible.
2. **Structured JSON logs** that include `trace_id`, service, version and tenant.
3. **A unified stack** - for example Prometheus/Mimir for metrics, Loki for logs, Tempo for traces, all in Grafana with exemplars and trace-to-logs links turned on.
4. **SLOs and dashboards per user journey**, with burn-rate alerts.

Example: A monitoring alert says "p99 latency > 2s". With observability you click the Grafana latency panel -> open the exemplar trace -> see the slow span is a DB query -> jump to the logs for that `trace_id` and see a lock timeout -> filter by `tenant` and find that only one large customer is affected.

What to say to the interviewer: *Monitoring tells me when something known is broken; observability lets me ask new questions about why, without shipping new code. For me that means high-cardinality context on every signal, metrics, traces and logs correlated through exemplars and trace IDs, and SLOs measured from the user's side with RUM and synthetics. I get there with OpenTelemetry instrumentation, structured logs and one Grafana stack over Prometheus, Loki and Tempo, so I can go from user pain to the exact failing request.*

Q3. What is the difference between logs, metrics and traces?

> **In simple words:** Metrics are your heart rate on a fitness watch, logs are a diary of everything you did today, and a trace is a GPS route of one single trip from home to office showing where you got stuck.

Detailed answer

	Metrics	Logs	Traces
What	Numbers over time	Timestamped text events	Path of one request across services
Example	`http_requests_total`, CPU 72%	`ERROR payment failed: timeout`	API -> auth -> orders -> DB, 1.8s total
Cost	Cheap, small	Expensive at volume	Medium, usually sampled
Best for	Alerting, trends, dashboards	Root-cause details	Finding which hop is slow
Tools	Prometheus, CloudWatch	Loki, ELK	Tempo, Jaeger, X-Ray

A common line: "Metrics show trends, logs tell stories, traces reveal the truth of a single request."

Example:

```
Metric: rate(http_requests_total{status=~"5.."}[5m])  -> 5xx rate jumped
Log:    {app="orders"} |= "ERROR"                        -> "DB connection refused"
Trace:  trace_id=ab12 -> orders span 4.9s -> db.query span 4.8s
```

> ***What to say to the interviewer:*** *Metrics are aggregated numbers I alert on, logs are detailed events I read to understand what happened, and traces follow one request across microservices to show which hop is slow. I usually start from a metric alert, use traces to find the slow service, and logs to find the exact error.*

Q4. What is log aggregation, and why do we need it?

> **In simple words:** Instead of visiting 200 shops to read each one's complaint book, you photocopy every complaint book into one central library where you can search them all at once.

Detailed answer

In Kubernetes, pods are temporary. When a pod dies, its local logs go with it, and with 100 pods you cannot `kubectl logs` each one. Log aggregation means:

1. **Collect** - an agent on every node (Fluent Bit, Promtail, Grafana Alloy, Fluentd, Vector) tails container logs from `/var/log/containers`.
2. **Enrich** - adds labels like namespace, pod, container, node.
3. **Ship** - sends to a central store (Loki, Elasticsearch, CloudWatch Logs).
4. **Store & retain** - with retention rules (e.g. 7 days hot, 90 days in S3).
5. **Search & alert** - query in Grafana/Kibana, alert on error patterns.

Example: LogQL query in Grafana for errors in one namespace:

```
{namespace="payments", app="checkout"} |= "ERROR" | json | status >= 500
```

> ***What to say to the interviewer:*** *Log aggregation collects logs from every server and pod into one central, searchable place so logs survive pod restarts and I can search across all services at once. A typical setup is a node-level agent like Fluent Bit or Promtail shipping into Loki or Elasticsearch, viewed in Grafana or Kibana.*

Q5. What are SLI, SLO, SLA and error budget?

> **In simple words:** A pizza shop measures how long delivery takes (SLI), sets an internal goal of 95% under 30 minutes (SLO), and promises customers a free pizza if it is over 45 minutes (SLA). The 5% of "late allowed" deliveries is the error budget.

Detailed answer

- **SLI (Service Level Indicator)** - the actual measurement. Example: percentage of requests that return success in under 300 ms.
- **SLO (Service Level Objective)** - the internal target for the SLI. Example: 99.9% of requests successful over 30 days.
- **SLA (Service Level Agreement)** - the external contract with penalties. Usually looser than the SLO so you get warned before breaking a contract.
- **Error budget** = 100% - SLO. For 99.9% over 30 days that is about 43 minutes of allowed "badness". If the budget is burning fast, you slow down risky releases and focus on reliability.

Availability	Downtime per 30 days
99%	~7.2 hours
99.9%	~43 minutes
99.99%	~4.3 minutes

Example: PromQL SLI for availability:

```
sum(rate(http_requests_total{status!~"5.."}[30d]))
/ sum(rate(http_requests_total[30d]))
```

> ***What to say to the interviewer:*** *The SLI is what I measure, the SLO is the internal target, and the SLA is the customer contract with penalties. The error budget is the allowed failure left over, and it becomes the rule for when we ship fast versus when we freeze features and fix reliability.*

Q6. How does Prometheus collect data from applications in Kubernetes?

> **In simple words:** Prometheus is a meter reader who walks door to door every 30 seconds and reads each house's meter. Kubernetes gives it the updated address list, so new houses get read automatically.

Detailed answer

1. Apps expose metrics in text format on an HTTP endpoint, usually `/metrics` (via a client library, or an **exporter** for things that cannot, e.g. node-exporter for hosts, postgres-exporter, kafka-exporter).
2. Prometheus uses **Kubernetes service discovery** to find pods/services/endpoints through the API server.

3. It **pulls (scrapes)** each target on an interval and stores samples in its time-series DB.
4. With the Prometheus Operator, you declare targets using a **ServiceMonitor** or **PodMonitor** instead of editing config files.
5. Short-lived jobs that end before being scraped push to a **Pushgateway**.
6. kube-state-metrics exposes object state (deployment replicas, pod phase), and cAdvisor in the kubelet exposes container CPU/memory.

Example:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: orders
  labels: { release: kps }      # must match the Prometheus selector
spec:
  selector:
    matchLabels: { app: orders }
  endpoints:
    - port: http-metrics
      path: /metrics
      interval: 30s
```

> ***What to say to the interviewer:*** *Prometheus uses a pull model - it discovers targets through the Kubernetes API and scrapes their /metrics endpoints on an interval. With the Prometheus Operator I just create a ServiceMonitor that selects the service by label, and exporters like node-exporter and kube-state-metrics cover infrastructure and cluster state.*

Q7. How do you monitor Kubernetes clusters and applications, and how do you integrate Prometheus and Grafana with Kubernetes?

Also asked as: How do you integrate monitoring tools like Prometheus and Grafana with Kubernetes? How do you monitor applications with Prometheus + Grafana (basic setup)?

> **In simple words:** Instead of building a hospital monitoring room piece by piece, you install a ready-made kit that comes with the heart monitors, the screens and the alarm bells already wired together.

Detailed answer

The standard way is the **kube-prometheus-stack** Helm chart. One install gives you:

- Prometheus Operator + Prometheus server
- Alertmanager
- Grafana with ready dashboards
- node-exporter (node CPU/mem/disk) and kube-state-metrics (object state)
- Default alert rules (KubePodCrashLooping, KubeNodeNotReady, etc.)

What to monitor, in layers:

- **Cluster/nodes** - CPU, memory, disk, pressure conditions, NotReady nodes.
- **Kubernetes objects** - restarts, pending pods, failed deployments, HPA at max.
- **Applications** - the RED method: Rate, Errors, Duration per service.
- **Logs** - Loki + Promtail/Alloy; **traces** - Tempo/Jaeger with OpenTelemetry.

Example:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install kps prometheus-community/kube-prometheus-stack -n monitoring --create-namespace
kubectl -n monitoring port-forward svc/kps-grafana 3000:80
```

Basic setup end to end, in 5 steps:

1. **Install the stack** - `helm install kps ...` as above (Prometheus, Alertmanager, Grafana ready).
2. **App exposes metrics** - add a Prometheus client library so the app serves `/metrics` (e.g. `http_requests_total`, `http_request_duration_seconds`), and name the port in the Service (e.g. `http-metrics`).
3. **Tell Prometheus to scrape it** - create a ServiceMonitor (see Q6) with the `release: kps` label; confirm the target is UP under Status -> Targets in the Prometheus UI.
4. **Grafana** - the Prometheus data source is pre-configured by the chart; build a RED dashboard (rate, errors, duration) or import a community dashboard by ID.
5. **Alert rule** - add a PrometheusRule; Alertmanager routes it to Teams/Slack/email.

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: orders-alerts
  labels: { release: kps }
spec:
  groups:
    - name: orders
      rules:
        - alert: OrdersHigh5xx
          expr: |
            sum(rate(http_requests_total{app="orders",status=~"5.."}[5m]))
            / sum(rate(http_requests_total{app="orders"}[5m])) > 0.05
          for: 5m
          labels: { severity: critical }
          annotations: { summary: "orders 5xx above 5% for 5 minutes" }
```

> ***What to say to the interviewer:*** *I install kube-prometheus-stack with Helm, which brings Prometheus, Alertmanager, Grafana, node-exporter and kube-state-metrics with default dashboards and rules. Then I add ServiceMonitors for each app, Loki for logs, and route Alertmanager to Teams or Slack by severity.*

Q8. What is a runbook, and what does on-call ownership mean?

> **In simple words:** A runbook is the "what to do if the fire alarm rings" card on the wall, so even a new person at 3 AM knows the exact steps without guessing.

Detailed answer

- A **runbook** is a step-by-step guide linked to an alert: what the alert means, how to check impact, commands to diagnose, how to fix or mitigate, and who to escalate to.
- Every page-worthy alert should have a runbook link in its annotations. If an alert has no clear action, it probably should not page anyone.

- **On-call ownership** means a rotating engineer is responsible for acknowledging alerts within an agreed time, mitigating, communicating and handing over cleanly. Good on-call has: a rotation schedule, escalation policy, primary + secondary, and a handover note.

Example: Alert rule with a runbook link:

```
- alert: KafkaConsumerLagHigh
  expr: sum by (group) (kafka_consumergroup_lag) > 50000
  for: 10m
  labels: { severity: critical }
  annotations:
    summary: "Consumer group {{ $labels.group }} lag is high"
    runbook_url: "https://wiki.example.com/runbooks/kafka-lag"
```

> ***What to say to the interviewer:*** *A runbook is a documented, step-by-step response for a specific alert, and I link it directly in the alert annotation. On-call ownership means someone is clearly accountable to acknowledge, mitigate, communicate and hand over, backed by a rotation and escalation policy.*

Q9. What is FinOps? What are the basics of cloud cost optimization?

> **In simple words:** FinOps is running cloud spending like a household budget: everyone can see the bill, each family member knows what they spend, and you stop paying for the gym you never visit.

Detailed answer

FinOps = shared responsibility for cloud cost between engineering, finance and business. The basic loop is **Inform -> Optimize -> Operate**.

- **Visibility** - tag every resource (team, env, project), use Cost Explorer / budgets / cost anomaly alerts.
- **Rightsizing** - match instance size and pod requests to real usage.
- **Pricing models** - Savings Plans/Reserved Instances for steady load, Spot for fault-tolerant load.
- **Waste removal** - unattached volumes, orphaned PVCs, idle load balancers, old snapshots, forgotten S3 buckets.
- **Storage tiering** - S3 lifecycle to Infrequent Access / Glacier; shorter log retention.
- **Data transfer** - NAT gateway and cross-AZ traffic are often the hidden cost; VPC endpoints help.

> ***What to say to the interviewer:*** *FinOps means making cloud cost visible and owned by the teams who create it, then continuously optimizing. My basics are tagging and budgets for visibility, rightsizing and Savings Plans or Spot for compute, lifecycle policies for storage, and regular audits for orphaned resources like unattached volumes and PVCs.*

Advanced

Q10. How do you implement centralized logging for microservices?

> **In simple words:** Every microservice writes in the same format and drops its letters into one shared post box, so anyone can search all letters by sender, date or tracking number.

Detailed answer

Design points:

1. **Standard log format** - JSON logs to stdout with fields: `timestamp`, `level`, `service`, `trace_id`, `request_id`, `message`. Do not write to files inside containers.
2. **Collector per node** - Fluent Bit / Promtail / Grafana Alloy as a DaemonSet reads `/var/log/containers/*.log` and adds Kubernetes labels.
3. **Central store** - Loki (cheap, label-indexed, object storage like S3) or Elasticsearch/OpenSearch (full-text search, heavier). CloudWatch Logs if you want fully managed on AWS.
4. **Keep labels low-cardinality** - namespace, app, env, level. Never put user IDs or request IDs as Loki labels; keep them inside the log line.
5. **Correlation** - include `trace_id` so Grafana can jump log -> trace.
6. **Retention & cost** - e.g. 7-30 days hot, then archive to S3; drop noisy debug logs at the agent.
7. **Security** - mask PII/secrets before shipping; RBAC on who can query which namespace.
8. **Alerting on logs** - Loki ruler or ElastAlert for patterns like sudden "OutOfMemory" or 5xx bursts.

Example:

```
Pods (stdout JSON) -> Fluent Bit DaemonSet -> Loki (S3 backend) -> Grafana
                                          \-> Loki ruler -> Alertmanager -> Teams
```

> ***What to say to the interviewer:*** *I standardize JSON logs to stdout with a trace ID, run a collector like Fluent Bit or Promtail as a DaemonSet, and ship into Loki backed by S3 or into OpenSearch. I keep labels low-cardinality, set retention and PII masking, and link logs to traces in Grafana.*

Q11. Design a highly available logging system for 100+ microservices across 3 regions.

> **In simple words:** Each region has its own local post office so mail never waits on a long-distance line, and there is one central search counter that can look into all three post offices.

Detailed answer

Architecture:

```
Region A/B/C:
  Pods -> Fluent Bit (DaemonSet, disk buffer)
       -> Kafka / Kinesis (regional buffer, optional)
       -> Loki (distributed mode, 3 AZs, replication_factor=3)
       -> S3 bucket in same region (chunks + index)
Global:
  Grafana (HA, 2+ replicas) with 3 Loki data sources
  or query federation -> single "all regions" view
```

Key decisions:

- **Regional ingestion** - logs stay in-region (lower latency, lower transfer cost, data residency rules).
- **Buffering** - agents with disk buffers; optionally Kafka in front so a Loki outage does not drop logs.
- **HA inside region** - Loki ingesters spread across 3 AZs with replication factor 3; stateless queriers/distributors behind a load balancer; object storage (S3) is durable by design.
- **Cross-region DR** - S3 cross-region replication of critical buckets.
- **Scale** - horizontal scaling of distributors/ingesters/queriers; rate limits per tenant so one noisy service cannot flood the cluster.
- **Multi-tenancy** - tenant per team or environment (`X-Scope-OrgID`).
- **Retention tiers** - short hot retention, long cold in S3 (with compliance needs).
- **Monitor the monitor** - alert on dropped logs, ingestion rate, agent errors.

Alternatives: OpenSearch with cross-cluster search, or a managed option (CloudWatch Logs, Datadog) if the team is small.

> ***What to say to the interviewer:*** *I would run a regional logging stack per region - Fluent Bit with disk buffers, optionally Kafka, then Loki in distributed mode across three AZs with S3 storage - and put a global HA Grafana on top that queries all three regions. That keeps ingestion local and resilient, uses tenant limits to stop noisy services, and adds cross-region replication only for logs we must keep for DR or compliance.*

Q12. How do you implement and monitor distributed tracing in microservices?

> **In simple words:** Tracing is like putting a tracking number on a parcel. Every warehouse it passes through scans it, so later you can see exactly which warehouse held it for three hours.

Detailed answer

1. **Instrument** services with **OpenTelemetry** SDKs or auto-instrumentation (Java agent, Python/Node auto-instrumentation) or eBPF tools like Grafana Beyla. Each request gets a `trace_id`; each hop is a **span**.
2. **Propagate context** through HTTP headers (W3C `traceparent`) and message headers (Kafka), so the trace does not break between services.
3. **Collect** with an **OpenTelemetry Collector** (agent per node + gateway) that batches, samples and exports.
4. **Store & view** in Tempo, Jaeger, or AWS X-Ray; view in Grafana.
5. **Sampling** - head sampling (keep 5-10%) or tail sampling (keep all errors and slow traces, sample the rest) to control cost.

6. **Correlate** - put `trace_id` in logs; enable exemplars so a latency metric links to a real trace.
7. **Monitor tracing health** - spans received vs dropped in the collector; alert if span rate falls to zero (a silently broken pipeline is common).

Example:

```
# OTel Collector tail sampling: keep errors + slow requests
processors:
  tail_sampling:
    policies:
      - name: errors
        type: status_code
        status_code: { status_codes: [ERROR] }
      - name: slow
        type: latency
        latency: { threshold_ms: 1000 }
```

> ***What to say to the interviewer:*** *I instrument services with OpenTelemetry, propagate the trace context through HTTP and Kafka headers, and send spans through an OTel Collector to Tempo or Jaeger viewed in Grafana. I use tail sampling to keep all errors and slow traces, put the trace ID in logs, and alert if span ingestion drops, because a silently broken trace pipeline is a common failure.*

Q13. How would you design an observability stack using CloudWatch, Prometheus and Grafana?

> **In simple words:** CloudWatch already watches the building AWS owns (RDS, ALB, Lambda), Prometheus watches what you run inside your own rooms (pods, apps), and Grafana is the single security desk showing both camera feeds.

Detailed answer

Split responsibilities:

- **CloudWatch** - AWS managed services: ALB (TargetResponseTime, HTTPCode_ELB_5XX), RDS (CPU, connections, FreeStorageSpace, replica lag), Lambda (errors, throttles, duration), MSK, ElastiCache, billing. Also CloudWatch Logs for Lambda/VPC flow logs.
- **Prometheus** - Kubernetes and apps: kube-state-metrics, node-exporter, app `/metrics`, exporters. Use Amazon Managed Prometheus or self-managed with remote_write to long-term storage (Thanos/Mimir) if retention is needed.
- **Grafana** - one place: CloudWatch data source (via IAM role, not access keys), Prometheus data source, Loki for logs, Tempo/X-Ray for traces.
- **Alerting** - Alertmanager for cluster/app alerts, CloudWatch Alarms -> SNS -> Lambda/chat for AWS services; or unify with Grafana Alerting. Route by severity and team.
- **Dashboards** - per-service RED dashboard, per-cluster USE dashboard (Utilization, Saturation, Errors), business/SLO dashboard.

Example:

```
EKS pods --scrape--> Prometheus --remote_write--> long-term store
AWS services --------> CloudWatch metrics/alarms --SNS--> Lambda -> Teams
Grafana <---- Prometheus + CloudWatch + Loki + Tempo data sources
```

What to say to the interviewer: *I use CloudWatch for AWS-managed services and Prometheus for Kubernetes and application metrics, and bring both into Grafana using an IAM role for the CloudWatch data source. Alerts come from Alertmanager and CloudWatch Alarms, routed by severity, with RED dashboards per service and an SLO dashboard on top.*

Q14. How would you monitor HPA scaling decisions in real time and detect if the metrics server is lagging?

In simple words: The HPA is a thermostat. If the thermometer on the wall is stuck showing an old temperature, the thermostat will make wrong decisions, so you watch both the thermostat's actions and the thermometer's freshness.

Detailed answer

What to watch:

- **HPA state** from kube-state-metrics: `kube_horizontalpodautoscaler_status_current_replicas`, `..._status_desired_replicas`, `..._spec_max_replicas`, and the `ScalingLimited` / `AbleToScale` conditions.
- **Events**: `kubectl describe hpa <name>` shows "FailedGetResourceMetric" or "unable to fetch metrics".
- **Metrics pipeline health**: `kubectl get apiservice v1beta1.metrics.k8s.io` must be Available; metrics-server pod restarts and its scrape latency; for custom metrics, prometheus-adapter or KEDA errors.
- **Freshness**: compare metric timestamps; metrics-server scrapes kubelets every ~15s by default. A lag appears as desired replicas not changing while real CPU climbs.

Example:

```
# Alert: HPA pinned at max for 15m (cannot scale further)
kube_horizontalpodautoscaler_status_current_replicas
  >= kube_horizontalpodautoscaler_spec_max_replicas

# Alert: desired != current for too long (scaling stuck)
kube_horizontalpodautoscaler_status_desired_replicas
  != kube_horizontalpodautoscaler_status_current_replicas

kubectl get --raw /apis/metrics.k8s.io/v1beta1/namespaces/live/pods | head
```

What to say to the interviewer: *I graph HPA current versus desired versus max replicas from kube-state-metrics and alert when it is pinned at max or when desired and current differ too long. For metrics-server lag I watch the metrics APIService availability, FailedGetResourceMetric events and metrics-server health, and for critical workloads I scale on faster signals like request rate or queue lag through KEDA.*

Q15. Your monitoring shows constant false alerts. How do you tune alerts and reduce alert fatigue?

Also asked as: How do you reduce alert noise? Reducing alert fatigue. How do you implement SLO-based alerting without alert fatigue (error budget, burn-rate alerts, multi-window)?

In simple words: If a car alarm goes off every time a leaf falls, people stop looking. You adjust it to ring only when someone actually tries to break in.

Detailed answer

Steps:

1. **Measure the noise** - list the top 10 alerts by count last month and how many led to action. Anything never actioned is a candidate to delete or downgrade.
2. **Alert on symptoms, not causes** - page on user impact (error rate, latency SLO burn, site down), not "CPU 80%" which might be normal.
3. **Add duration** - `for: 5m` or 10m so brief spikes do not fire.
4. **Better thresholds** - base them on historical percentiles, not guesses; use SLO burn-rate alerts (fast burn pages, slow burn tickets).
5. **Deduplicate and group** - Alertmanager `group_by`, inhibition rules (if the node is down, suppress all pod alerts on it).
6. **Severity routing** - critical -> page/phone; warning -> chat channel; info -> dashboard only.
7. **Silence during maintenance** windows.
8. **Every page must have a runbook and an owner**; review alerts in a weekly or monthly ops review.
9. **Check the alerting path too** - false negatives (alerts silently not delivered) are as dangerous as false positives.

SLO-based alerting with multi-window, multi-burn-rate rules - burn rate is how fast you are using the error budget (1x = the budget lasts exactly the 30-day window). Each rule needs both a long window (proves it is significant) and a short window (proves it is still happening, so the alert clears quickly after recovery). Recommended values for a 99.9% SLO:

Burn rate	Long window	Short window	Budget used	Action
14.4x	1h	5m	2%	Page
6x	6h	30m	5%	Page
1x	3d	6h	10%	Ticket

This alerts on a symptom (users getting errors), not on causes like CPU, and it sends fast burns to the pager and slow burns to a ticket queue. For the PromQL of the 14.4x rule and how it fits SLA reporting, see Q17.

Example:

```
route:
  group_by: [alertname, cluster, namespace]
  group_wait: 30s
  repeat_interval: 4h
inhibit_rules:
  - source_matchers: [alertname="KubeNodeNotReady"]
    target_matchers: [severity="warning"]
    equal: [node]
```

What to say to the interviewer: *I first measure which alerts fire most and which ones actually led to action, then delete or downgrade the noise. I alert on user-facing symptoms with a for-duration, use SLO burn-rate alerts, group and inhibit related alerts in Alertmanager, route by severity, and make*

sure every paging alert has a runbook and an owner.

Q16. You are asked to design an alerting system for high availability. What is your approach?

In simple words: A fire alarm system that itself can fail silently is useless, so you install two alarm panels, two phone lines, and a test button you press regularly.

Detailed answer

Design:

- **Redundant alert evaluation** - two Prometheus replicas scraping the same targets (HA pair), or a managed service.
- **Alertmanager cluster** - 3 replicas in gossip mode; they deduplicate so you get one notification, not three.
- **Multiple notification channels** - primary (pager/phone via on-call tool) plus secondary (Teams/Slack, email). Critical alerts should not depend on a single webhook.
- **Watchdog / dead man's switch** - an always-firing alert (`vector(1)`) sent to an external service; if it stops arriving, the external service alerts you that your monitoring is dead.
- **External synthetic checks** - uptime checks from outside your cloud (another region/account/provider), so a total outage of your cluster still alerts.
- **Escalation policy** - primary -> secondary -> manager, with acknowledge timeouts.
- **Test end-to-end regularly** - send test alerts and confirm they arrive in the real channel; verify templates render (a broken message template can silently block delivery).
- **Alert as code** - rules in Git, reviewed and deployed through CI.

Example:

```
- alert: Watchdog
  expr: vector(1)
  labels: { severity: none }
  annotations:
    summary: "Heartbeat - if this stops, the alerting pipeline is broken"
```

What to say to the interviewer: *I make every layer redundant - HA Prometheus, a three-node Alertmanager cluster and more than one notification channel - and add a dead man's switch plus external synthetic checks so we find out if monitoring itself dies. Alert rules live in Git, escalation is defined, and I test delivery end to end regularly because a silently broken notification path is worse than a noisy one.*

Q17. How would you monitor the end-to-end SLA for services involved in a payments pipeline?

Also asked as: How would you monitor SLA compliance for a business-critical service?

In simple words: A customer does not care that each counter in the bank was fast; they care how long it took from walking in to getting the receipt. So you time the whole journey, not just each counter.

Detailed answer

1. **Define the user journey** - e.g. "payment initiated -> authorized -> captured -> confirmation shown". The SLI is end-to-end success and latency of that journey.
2. **SLIs per step and overall**: success rate, p95/p99 latency, and for async steps the **freshness/lag** (time from event produced to processed).
3. **Distributed tracing** across gateway, payment service, fraud check, bank adapter, queue consumers - with the same trace ID through Kafka headers.
4. **Business metrics** - payments per minute, success ratio by bank/method; a sudden drop in successful payments is often the best alert.
5. **Synthetic transactions** - a test card payment every minute from outside.
6. **Third-party dependency monitoring** - track bank/PSP response time and error codes separately so you know if it is "us or them".
7. **SLO + error budget + burn-rate alerts** on the journey SLI; dashboards showing budget remaining.
8. **Reconciliation checks** - count of initiated vs completed; stuck payments older than X minutes.

For SLA compliance of any business-critical service, add:

- **Multi-window, multi-burn-rate SLO alerts** - page when the error budget burns fast over both a long and a short window (for example 14.4x over 1h and 5m), and open a ticket for a slow burn (for example 6x over 6h and 30m). This catches real incidents quickly without paging for tiny blips.
- **Synthetic probes from outside** your cloud and region (for example the Blackbox exporter or an external uptime checker), so you measure what customers see even if your whole cluster is down.
- **Monthly SLA reports** - availability percentage, downtime minutes per service, and the root cause of each breach, compared with the contract target.
- **Error budget policy** - agreed in writing: when the budget is spent, risky releases freeze and reliability work comes first.

```
# Fast-burn page for a 99.9% SLO (budget = 0.001)
( sum(rate(http_requests_total{job="payments",status=~"5.."}[1h]))
  / sum(rate(http_requests_total{job="payments"}[1h])) > 14.4 * 0.001 )
and
( sum(rate(http_requests_total{job="payments",status=~"5.."}[5m]))
  / sum(rate(http_requests_total{job="payments"}[5m])) > 14.4 * 0.001 )
```

Example:

```
# End-to-end success ratio (5m)
sum(rate(payment_completed_total{status="success"}[5m]))
/ sum(rate(payment_initiated_total[5m]))
# Alert if stuck payments > 0 for 10m
payments_pending_older_than_5m > 0
```

> ***What to say to the interviewer:*** *I define the SLI on the whole payment journey, not individual services, and measure it with business metrics, distributed traces through every hop including Kafka, and synthetic test payments. I alert on SLO burn rate and on stuck or unreconciled payments, and track third-party banks separately so we know quickly whether the problem is ours or theirs.*

Q18. How would you design auto-scaling for 50M+ concurrent viewers across multiple Kubernetes clusters without over-provisioning?

In simple words: For a huge cricket match, a stadium does not keep 50 lakh seats open all year. It knows the match schedule, opens extra gates just before the crowd arrives, and closes them after, while always keeping a small safety margin.

Detailed answer

Layers of scaling:

1. **Push most traffic to the CDN** - video segments are served from CDN edges; Kubernetes serves manifests, auth, session, entitlement, ads APIs. Offloading is the biggest cost saver.
2. **Scale on the right signals** - not only CPU. Use requests per second, concurrent sessions, or Kafka/queue lag via **KEDA** or custom metrics HPA.
3. **Predictive/scheduled scaling** - events are known in advance; use KEDA cron scaler or scheduled scaling to raise `minReplicas` before the toss, based on last event's peak + growth forecast.
4. **Fast node provisioning** - Karpenter (or Cluster Autoscaler) with multiple instance types and Spot for stateless tiers, On-Demand for critical tiers.
5. **Headroom without waste** - low-priority "placeholder" (overprovisioning) pods that reserve capacity and get evicted instantly when real pods need room.
6. **Multi-cluster distribution** - several regional clusters behind global load balancing; each cluster scales independently, and a cell-based design limits blast radius.
7. **Protect the system** - rate limits, load shedding, graceful degradation (drop to lower bitrate, disable non-critical features).
8. **Scale down carefully** - stabilization windows so it does not flap during innings breaks.

Example:

```
# KEDA: pre-scale before match + scale on concurrent sessions
triggers:
  - type: cron
    metadata: { timezone: Asia/Kolkata, start: "0 18 * * *", end: "0 1 * * *",
                desiredReplicas: "400" }
  - type: prometheus
    metadata:
      serverAddress: http://prometheus.monitoring:9090
      query: sum(active_sessions{svc="playback-api"})
      threshold: "5000"
```

What to say to the interviewer: *I push video delivery to the CDN, then scale the Kubernetes APIs on business signals like concurrent sessions using KEDA, combined with scheduled pre-scaling before known events. Karpenter with mixed Spot and On-Demand adds nodes quickly, low-priority placeholder pods give instant headroom, and multiple regional clusters, rate limits and graceful degradation protect us without paying for peak capacity all year.*

Q19. During an IPL final a new region needs to spin up instantly. How would you pre-warm nodes and scale workloads with zero cold-start impact?

In simple words: Before a big wedding, the caterer starts the stoves and cooks some food in advance, so when guests arrive they do not wait for the gas to be lit.

Detailed answer

Cold starts come from: new nodes booting, image pulls, app warm-up (JVM, caches), empty caches, and load balancers/CDN not warmed. Remove each one:

1. **Infrastructure ready in advance** - the region is already built by Terraform (same modules), cluster running at a small baseline. Standing up from nothing on the day is too risky.
2. **Pre-warm nodes** - raise the node group minimum or use Karpenter with placeholder (pause) pods at low priority 1-2 hours before; request capacity reservations for critical instance types.
3. **Pre-pull images** - a DaemonSet that pulls the big images, or bake images into the AMI; keep images small; use a registry replica in that region.
4. **Pre-scale pods** - raise `minReplicas` on HPA/KEDA ahead of time.
5. **App warm-up** - startup probes and readiness probes so pods only receive traffic after warm-up; warm caches (Redis) with popular content/metadata.
6. **Warm the edge** - CDN pre-fetch of manifests/assets; ask AWS to pre-warm load balancers for massive expected jumps if needed.
7. **Shift traffic gradually** - weighted routing (5% -> 25% -> 100%) while watching SLOs.
8. **Rehearse** - load test with k6/Locust at expected peak in the days before.

What to say to the interviewer: *I would never build the region on the day - it is pre-created from the same Terraform modules and running at a baseline. Before the match I pre-scale nodes with placeholder pods or capacity reservations, pre-pull images, raise minReplicas, warm caches and CDN, gate traffic on startup and readiness probes, and shift traffic in weighted steps after a rehearsed load test.*

Q20. How would you use Envoy and Istio to route low-latency live streams differently from VOD without service restarts?

In simple words: At a toll plaza, the traffic police can change which lane ambulances use just by changing the signboards, without closing the road or rebuilding the plaza.

Detailed answer

- Istio puts an **Envoy sidecar** (or ambient-mode proxy) next to each service. Envoy receives its routing config dynamically from **istiod** over the xDS API, so changing routes does not restart pods.
- Use a **VirtualService** to match live vs VOD requests (by path, header, or host) and send them to different subsets or services.
- Use a **DestinationRule** to give each subset different traffic policies: tight timeouts, fewer retries (retries add latency for live), connection pool limits, outlier detection, and locality-aware load balancing for live; more generous timeouts and caching-friendly settings for VOD.

Example:

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: { name: playback }
spec:
  hosts: [playback.svc.cluster.local]
  http:
    - match: [{ uri: { prefix: /live/ } }]
      route: [{ destination: { host: playback-live } }]
      timeout: 2s
      retries: { attempts: 1, perTryTimeout: 1s }
    - route: [{ destination: { host: playback-vod } }]
      timeout: 10s
      retries: { attempts: 3, perTryTimeout: 3s }
```

Apply with `kubectl apply`; Envoy picks it up in seconds. Roll out changes safely with weights (90/10) and watch latency.

> ***What to say to the interviewer:*** *Envoy gets its config dynamically from istiod over xDS, so I can change routing just by applying Istio resources with no restarts. A VirtualService matches live versus VOD by path or header and sends them to different backends, and DestinationRules give live traffic tight timeouts, minimal retries, locality-aware balancing and outlier detection, while VOD gets more relaxed settings.*

Q21. How would you configure readiness and liveness probes to catch buffering or lag in stream-processing services before users notice?

> **In simple words:** Do not just check if the worker is breathing (liveness); check if they are keeping up with the work (readiness). A worker who is alive but far behind should stop getting new work.

Detailed answer

(Probe basics are covered in the Kubernetes chapter; this is the streaming angle.)

- **Readiness** should reflect "can I serve good quality right now": check internal lag (e.g. consumer lag or segment processing delay below a threshold), dependency reachability (Redis, origin). If lag is high, return 503 so the pod leaves the load balancer and traffic goes to healthy pods.
- **Liveness** should be simple and only detect a truly stuck process (event loop hung, no progress for N minutes). Do not put dependency checks in liveness or you will restart every pod when Redis blips.
- **Startup probe** protects slow warm-up.
- Probes are not enough - also alert on business signals: rebuffering ratio, segment generation delay, consumer lag.

Example:

```
readinessProbe:
  httpGet: { path: /ready, port: 8080 }   # returns 503 if lag > 2s
  periodSeconds: 5
  failureThreshold: 2
livenessProbe:
  httpGet: { path: /live, port: 8080 }    # 500 only if no progress for 60s
  periodSeconds: 10
  failureThreshold: 6
```

What to say to the interviewer: *I make readiness reflect real serving quality, like processing lag under a threshold, so a lagging pod is taken out of rotation before users buffer, while liveness only detects a truly stuck process so we do not cause restart storms. Alongside probes I alert on rebuffering ratio and consumer lag, because probes only protect one pod at a time.*

Q22. How do you build a culture where latency SLOs are enforced like uptime SLAs?

In simple words: A restaurant that only checks "is the door open" will miss that customers wait 40 minutes for food. You make "food served within 15 minutes" a goal everyone is measured on, not just "open for business".

Detailed answer

1. **Make latency a first-class SLO** - e.g. "99% of play starts under 2 seconds" and "rebuffer ratio under 0.5%", agreed with product owners, because slow is the new down for streaming.
2. **Visible dashboards** - SLO and error budget shown on team dashboards and in weekly reviews.
3. **Error budget policy** - written and agreed: if the latency budget is exhausted, feature releases pause and reliability work gets priority.
4. **Alerts on burn rate** of latency SLOs, not just on up/down.
5. **Shift left** - performance tests in CI, latency budgets per service in design reviews, canary analysis that rolls back if p99 regresses.
6. **Ownership** - each service team owns its SLO; postmortems for latency incidents just like outages.
7. **Celebrate improvements** and keep it blameless.

What to say to the interviewer: *I define latency SLOs with product teams, show them and their error budgets on shared dashboards, and alert on burn rate exactly like availability. A written error budget policy pauses feature work when the budget is exhausted, canary analysis rolls back latency regressions automatically, and latency incidents get the same blameless postmortems as outages.*

Q23. How would you simulate chaos in a streaming pipeline without risking real user impact?

In simple words: Fire drills are done with a plan, a small area first, and a way to stop immediately - not by setting the real building on fire during a wedding.

Detailed answer

1. **Start with a hypothesis** - "If one Kafka broker dies, consumer lag stays below 30s and no playback errors".
2. **Start in staging/pre-prod** with production-like load (replayed or synthetic traffic).
3. **In production, limit blast radius** - one pod, one AZ, a canary cell, or only synthetic/test users; never during a live event.
4. **Guardrails / abort conditions** - automatic stop if SLO metrics cross a threshold.
5. **Tools** - Chaos Mesh or LitmusChaos in Kubernetes (pod kill, network delay, packet loss), AWS Fault Injection Service for AZ/instance failures, Istio fault injection for latency on specific

routes or headers only.

6. **Shadow traffic** - mirror real traffic to a copy of the pipeline and break the copy.
7. **Game days** - scheduled, announced, with the on-call team; document findings and fix gaps.

Example:

```
# Istio fault injection only for requests with a test header
http:
  - match: [{ headers: { x-chaos-test: { exact: "true" } } }]
    fault: { delay: { percentage: { value: 100 }, fixedDelay: 3s } }
    route: [{ destination: { host: playback-live } }]
```

> ***What to say to the interviewer:*** *I run chaos as controlled experiments with a clear hypothesis, starting in staging with production-like load, then in production only on a small blast radius such as one canary cell or synthetic test traffic, never during live events. I use tools like Chaos Mesh, AWS FIS or Istio fault injection with automatic abort conditions on SLO metrics, and turn every finding into a fix.*

Q24. How do you justify infrastructure costs for pre-warmed scaling capacity to executives before a major sports event?

> **In simple words:** You explain it like insurance: "We will spend 10 lakh for these three days so we do not risk losing 5 crore in ads and subscribers if the app crashes in the final over."

Detailed answer

Talk in business terms, not CPU:

1. **Cost of failure** - revenue per minute of the event (ads, subscriptions, sponsors), churn risk, brand damage, SLA penalties. Use past incidents as evidence.
2. **Data-backed forecast** - last year's peak concurrency, growth trend, and the gap between how fast autoscaling reacts (minutes) vs how fast traffic jumps (seconds at toss/final over).
3. **Exact, time-boxed ask** - pre-warm only for event windows, with a scale-down plan; show the cost per hour.
4. **Options table** - no pre-warm (cheapest, highest risk), partial pre-warm, full pre-warm - with cost and risk for each.
5. **Cost reducers** - Spot for stateless tiers, capacity reservations only for peak hours, CDN offload, Savings Plans for the baseline.
6. **Show results after** - report actual peak, cost, and SLOs met, which builds trust for the next ask.

Option	Extra cost (event days)	Risk
Reactive autoscaling only	Lowest	High - cold start at toss
Partial pre-warm (60% of peak)	Medium	Medium
Full pre-warm + Spot for stateless	Higher	Low

> ***What to say to the interviewer:*** *I compare the time-boxed cost of pre-warming against the business cost of failure - lost ad revenue, churn and penalties per minute of downtime - using last event's real peak data. I present two or three options with cost and risk, reduce the bill with Spot, reservations*

only for the event window and CDN offload, and report actual results afterwards.

Scenario-Based

Q25. "Production is down. Users are impacted. What will you do?"

Also asked as: How do you debug a production issue? What is your general troubleshooting approach?

In simple words: When a patient arrives bleeding, the doctor first stops the bleeding, then finds the cause. You restore service first, investigate deeply later.

Detailed answer

1. **Acknowledge and declare** - take ownership, open an incident channel/bridge, assign roles: Incident Commander, Tech lead, Comms. Set severity.
2. **Assess impact** - which users, regions, features? Check SLO dashboards, error rate, synthetic checks, status of dependencies (cloud status page).
3. **Ask "what changed?"** - recent deploys, config changes, infra changes, feature flags, certificate expiry, traffic spike. Most incidents follow a change.
4. **Mitigate first** - rollback the last deploy, disable a feature flag, fail over, scale up, restart a stuck component, block abusive traffic. Stop the bleeding before root cause.
5. **Diagnose in layers** - edge (DNS, CDN, LB) -> ingress -> service -> dependencies (DB, cache, queue) using metrics, then traces, then logs.
6. **Communicate** regularly - every 15-30 minutes to stakeholders and status page, even if "still investigating".
7. **Verify recovery** - metrics back to normal, synthetic checks green, users confirm.
8. **Postmortem** - blameless RCA, action items with owners and dates.

General troubleshooting methodology (the 10 rules I follow for any issue):

1. **Check logs first** - application, pod events, system logs; the error message often names the cause.
2. **Trace the request flow** - user -> DNS -> CDN/LB -> ingress -> service -> pod -> DB/cache; find the first hop that fails.
3. **Process of elimination** - test one layer at a time (curl the service directly, then the pod, then the DB) and cross off what works.
4. **Infra vs app** - are all services affected (likely infra: node, network, DNS) or only one (likely app: code, config)?
5. **Validate external dependencies** - third-party APIs, cloud provider status, certificates, DNS, managed DB/cache.
6. **Check resource limits** - CPU throttling, OOMKilled, disk full, connection pools, file descriptors, quotas.
7. **Reproduce in a test environment** - confirm the cause and test the fix safely before touching production.
8. **Keep a known-issues doc / runbook** - check it first, and add every new issue and fix to it afterwards.

9. **Use health checks** - readiness/liveness probes, LB target health and synthetic checks show where it breaks.
10. **Know when to escalate** - if impact is growing or the issue is outside your area, pull in the owner early; time-box solo debugging.

Example: First 5 commands in a Kubernetes incident:

```
kubectl get pods -A | grep -v Running
kubectl get events -A --sort-by=.lastTimestamp | tail -30
kubectl rollout history deploy/checkout -n prod
kubectl logs deploy/checkout -n prod --since=15m | grep -i error | tail
kubectl rollout undo deploy/checkout -n prod     # if the last deploy is the cause
```

> ***What to say to the interviewer:*** *I declare an incident, set up roles and a channel, and quickly assess the blast radius. I check what changed recently and mitigate first - usually a rollback, feature flag or failover - then diagnose layer by layer with metrics, traces and logs, keep stakeholders updated on a fixed cadence, confirm recovery, and follow up with a blameless postmortem.*

Q26. Monitoring shows increased response time, but infrastructure (CPU, memory, pods) looks healthy. How would you investigate?

> **In simple words:** The kitchen staff are not tired and the stoves are working, but food is still late. Maybe the delivery boy is stuck in traffic, or they are waiting for a supplier - the problem is outside the kitchen.

Detailed answer

Healthy CPU/memory usually means the service is **waiting**, not working. Look for:

1. **Where is the time spent?** - open a slow trace: which span is slow - DB, cache, external API, another service?
2. **Dependencies** - DB slow queries, lock waits, connection pool exhaustion (threads waiting for a connection), Redis latency, third-party API latency.
3. **Saturation that is not CPU** - thread pools, connection pools, file descriptors, network bandwidth, disk IOPS/EBS burst credits, CPU throttling from low limits (`container_cpu_cfs_throttled_seconds_total`) even when node CPU looks fine.
4. **Scope it** - one endpoint, region, AZ, customer or version? Compare p50 vs p99 (only tail = contention/GC/noisy neighbor).
5. **Network** - DNS lookup delays, TLS handshakes, cross-AZ hops, retries/timeouts, load balancer queueing.
6. **Recent changes** - deploys, config, a new feature causing N+1 queries, bigger payloads.
7. **GC pauses** in JVM/Node apps.
8. **Time pattern and traffic level** - if the slowness is periodic or happens off-peak, suspect scheduled work (cron, backups, snapshots, DB maintenance) or cold starts after scale-down (empty caches, closed connections, cold Lambdas). At low traffic a few slow requests can distort p99, so check the request count before trusting the percentile.

Example:

```
# Is the pod CPU-throttled even though node CPU is fine?
rate(container_cpu_cfs_throttled_periods_total{pod=~"orders.*"}[5m])
/ rate(container_cpu_cfs_periods_total{pod=~"orders.*"}[5m])

# p99 by endpoint to scope the problem
histogram_quantile(0.99, sum by (le, route)
  (rate(http_request_duration_seconds_bucket{app="orders"}[5m])))
```

> ***What to say to the interviewer:*** *If CPU and memory are fine, the service is usually waiting on something, so I open slow traces to see which dependency or hop the time is spent in. I check database slow queries and connection pools, cache and third-party latency, CPU throttling from limits, thread pools, DNS and network, scope it by endpoint or region, and correlate it with recent changes.*

Q27. It is 2 AM. Users report the app is slow. There are no errors and no logs. What is your first move?

Also asked as: After a deploy, latency spikes for 30% of users - no errors, no logs. What now? You get paged at 2 AM for high latency across services - what is your move? (tests calm under pressure)

> **In simple words:** A customer calls at night saying the shop is slow, but the cashier's notebook is empty. Either the customers never reached the counter, or the notebook itself stopped working - so first check the door and the notebook, then the kitchen.

Detailed answer

1. **Confirm and scope** - synthetic checks and real-user monitoring (RUM): is it real, all users or some, one region/endpoint? Compare latency **p50 vs p95/p99** against the error rate (slow with zero errors = something is waiting, not crashing).
2. **Golden signals / RED per service** - rate, errors, duration, saturation. Which service's latency moved first?
3. **Traces** - open a slow trace (or a latency exemplar) and find the slow span.
4. **Dependencies** - DB slow queries, locks, connection pool saturation, cache hit ratio drop, third-party API latency.
5. **What runs at 2 AM?** - backups, cron jobs, batch ETL, DB autovacuum, log rotation, EBS/RDS snapshots. Night-time slowness is very often scheduled work.
6. **Noisy neighbours and GC** - another workload on the same node or DB, burst credits exhausted, long GC pauses.
7. **"No logs" is itself a clue:**

- the **log pipeline may be down** (agent crashed, Loki/ELK ingestion stuck, disk full), so check whether any service is logging right now; or
- **requests are not reaching the app at all**, so look before the app: load balancer/CDN metrics and queueing, DNS resolution, TLS handshake or certificate problems.

8. **Mitigate first** - scale out, fail over, pause or reschedule the batch job, then confirm latency recovers.
9. **Communicate** - short update to stakeholders or the status page, then a proper RCA the next day.

The "paged at 2 AM, latency high across many services" variant (calm under pressure):

- **Acknowledge the page** right away so nobody else is left wondering, then **set severity** based on user impact.
- **Open an incident channel** and write down what you see and when; a calm, written process beats panicked clicking.
- **Many services slow at once = look for one shared dependency** - a database, DNS/CoreDNS, the network or a NAT/cross-AZ path, a shared cache (Redis), the ingress/load balancer, or a noisy node pool where all these pods run together.
- **Mitigate before root-causing** - fail over, scale out, cordon the bad nodes, pause the batch job.
- **Post updates on a fixed cadence** (for example every 15-30 minutes), even if it is only "still investigating".
- **Escalate early** to the owner of the shared dependency; escalating early is a strength, not a weakness.

The post-deploy variant (30% of users slow after a release):

- **Correlate with the deploy timestamp** - Grafana deploy annotations make this obvious on the latency graph.
- **30% is a strong clue** - it usually means a subset: canary pods on the new version, one AZ, one node group, or one ingress path/route.
- **Compare latency by label** - `version`, `pod`, `zone`, `route`; open trace exemplars from the slow series to find the slow span in the new code (new query, missing index, bigger payload, lower resource limits causing CPU throttling).
- **Roll back first if the SLO is burning** - `kubectl rollout undo` / `helm rollback`, or shift canary weight to 0, then debug the new version off the critical path.

Example:

```
# Tail-only or everyone? Compare percentiles
histogram_quantile(0.50, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
# Post-deploy: p99 split by version (old vs new pods)
histogram_quantile(0.99, sum by (le, version)
  (rate(http_request_duration_seconds_bucket{app="checkout"}[5m])))
# Are logs flowing at all? (LogQL)
sum by (namespace) (count_over_time({namespace=~".+"}[5m]))
kubectl get cronjobs -A          # anything scheduled at 2 AM?
```

> ***What to say to the interviewer:*** *First I confirm and scope it with synthetic checks, RUM and p50 versus p99, then follow RED metrics and traces to the slow span, and check dependencies and anything scheduled at that hour like backups, ETL or vacuum. No logs is itself a clue - either the log pipeline is broken or requests are not reaching the app, so I check the LB, CDN, DNS and TLS. If it started after a deploy and hits about 30% of users, I compare latency by version, pod and AZ and roll back first if the SLO is burning.*

Q28. Application metrics are not being scraped by Prometheus. What could cause it, and how do you validate scraping?

In simple words: The meter reader is not recording your house. Either he does not have your address, he is going to the wrong door, the gate is locked, or there is no meter on the wall. Check each one in order.

Detailed answer

Common causes:

- **Target not discovered** - ServiceMonitor labels do not match the Prometheus `serviceMonitorSelector` (for example a missing `release: kps` label), or its `namespaceSelector` does not include the app's namespace.
- **Selector mismatch** - the ServiceMonitor `selector.matchLabels` does not match the Service's labels.
- **Port name mismatch** - the ServiceMonitor `port:` must be the named port in the Service (for example `http-metrics`), not the number.
- **Wrong path or scheme** - the app serves `/actuator/prometheus` or HTTPS, but the scrape uses `/metrics` over HTTP -> `server returned HTTP status 404`.
- **Auth** - `401/403` because the endpoint needs a token or basic auth.
- **Network** - a NetworkPolicy blocks the monitoring namespace, or the app binds to `127.0.0.1` so it is unreachable from outside the pod.
- **App does not expose metrics** at all (no client library or exporter).
- **Prometheus RBAC** - no permission to list/watch endpoints or pods in that namespace.
- **relabel_configs** dropping the target, or `scrape_timeout` too low -> `context deadline exceeded`.

How to validate, in order:

1. Prometheus UI -> **Status -> Service Discovery**: is the target discovered or dropped?
2. **Status -> Targets**: is it up or down, and what is the error text?
3. Query `up{job="orders"}` - 1 = scraped OK, 0 = scrape failing, no result = not discovered.
4. Test the endpoint directly from outside the pod.

Example:

```
kubectl -n prod port-forward svc/orders 8080:8080
curl -s localhost:8080/metrics | head          # does the app expose metrics?
kubectl -n prod get svc orders -o yaml | grep -A5 ports    # check the port NAME
kubectl -n monitoring get prometheus -o yaml | grep -A3 serviceMonitorSelector
kubectl -n prod get servicemonitor orders -o yaml
# PromQL
up{job="orders"}
```

What to say to the interviewer: *I check the Prometheus Service Discovery and Targets pages and the up metric to see whether the target is undiscovered or failing, and read the exact error. The usual causes are ServiceMonitor labels or namespace not matching the Prometheus selector, a port-name mismatch, a wrong path or scheme, auth errors, a NetworkPolicy, the app binding to localhost, RBAC or relabeling. I confirm the fix with a port-forward and curl on /metrics, then check that up returns 1.*

Q29. Playback failures spike for only 3% of users in the APAC region. CPU, memory and pods look fine. Walk through your triage plan.

In simple words: If only some customers in one city complain about a shop, the shop itself is probably fine - the problem is one road, one delivery van, or one type of customer.

Detailed answer

A small, regional slice means **something shared by that 3%**, not a general capacity issue.

1. **Confirm and size it** - error rate and play-start failures by region, ISP, CDN edge/PoP, device, OS, app version, content ID, DRM type. Real-user monitoring (RUM) / client telemetry is key because servers may look fine.
2. **Find the common factor** - one CDN PoP, one ISP, one AZ, one app version, one codec, one content package?
3. **Edge and network** - CDN edge health and cache hit ratio in APAC, origin errors from that PoP, TLS certificate chain issues on older devices, DNS resolution for that ISP, BGP/routing issues.
4. **Application paths** - DRM/license server errors, entitlement/geo-blocking rules, token expiry or clock skew, a canary or feature flag rolled out only to APAC or to 3% of users (a 3% canary is a big clue).
5. **Recent changes** - CDN config, new app release on one platform, certificate rotation.
6. **Mitigate** - route that region/ISP to another CDN or PoP, roll back the canary/flag, purge bad cache objects.
7. **Follow up** - add alerts sliced by region/ISP/version so a 3% issue is not hidden inside a global average.

What to say to the interviewer: *When only 3% of one region fails while servers look healthy, I look for what those users share, slicing client telemetry by CDN PoP, ISP, device, app version, content and DRM type. Typical culprits are one CDN edge, an ISP routing or DNS problem, a certificate issue on old devices, or a canary rolled out to a small percentage, and I mitigate by shifting CDN or rolling back, then add region and version-sliced alerts.*

Q30. Your Kafka ingestion pipeline lags by 2 minutes during a traffic surge. Producers are fine, consumers are idle. What is your debug path?

In simple words: Parcels are piling up at the warehouse, and the delivery boys are sitting idle. So they are either not assigned any parcels, cannot reach the parcels, or are waiting for permission - the problem is in the assignment, not the workload.

Detailed answer

"Lag high + consumers idle" means consumers are not actually fetching or processing. Check in order:

1. **Consumer group state** - is it stuck **rebalancing**? Frequent rebalances (members joining/leaving, `max.poll.interval.ms` exceeded, pod restarts, HPA scaling consumers up and down) stop all consumption.
2. **Partition assignment** - more consumers than partitions leaves extras idle; all lag may be on a few **hot partitions** (bad key choice) assigned to one consumer while others sit idle.

3. **Consumer blocked downstream** - waiting on DB/Redis/API calls, so the thread is blocked (looks idle on CPU) - check traces and connection pools.
4. **Fetch configuration** - `fetch.min.bytes`, `fetch.max.wait.ms`, `max.poll.records` too small; consumer paused due to backpressure.
5. **Broker side** - broker CPU/disk/network, under-replicated partitions, leader imbalance, throttling quotas, broker disk full.
6. **Connectivity/auth** - consumers cannot reach new partition leaders, ACL/cert errors in consumer logs.
7. **Autoscaling** - scale consumers on lag with KEDA, but only up to the partition count; add partitions carefully (changes key ordering).

Example:

```
kafka-consumer-groups.sh --bootstrap-server $BROKER --describe --group ingest-svc
# Shows per-partition CURRENT-OFFSET, LOG-END-OFFSET, LAG, CONSUMER-ID
# Empty CONSUMER-ID with high LAG  -> partition not assigned
# All lag on 2 partitions          -> hot key / skew
kafka-consumer-groups.sh --bootstrap-server $BROKER --describe --group ingest-svc --state
# State: PreparingRebalance / CompletingRebalance -> rebalance storm
```

> ***What to say to the interviewer:*** *Idle consumers with growing lag tells me they are not fetching, so I describe the consumer group to check for rebalance storms, unassigned partitions, hot partitions from key skew, and more consumers than partitions. Then I check whether consumers are blocked on a downstream dependency, their fetch and poll settings, and broker health, and I scale consumers on lag with KEDA only up to the partition count.*

Q31. Sudden tail latency on the Redis-based stream session store during a Champions League match. How do you find and fix the bottleneck?

> **In simple words:** Redis is one very fast cashier who serves one customer at a time. If one customer asks to count every coin in the shop, everyone behind them waits - that is tail latency.

Detailed answer

Redis executes commands on a **single thread**, so one slow command or hot key hurts everyone's p99.

1. **Confirm it is Redis** - traces show slow Redis spans; compare client-side vs server-side latency (network vs Redis itself).
2. **Slow commands** - `SLOWLOG GET`; look for `KEYS *`, large `HGETALL`, `SMEMBERS`, `LRANGE 0 -1`, big `DEL` on huge keys, Lua scripts.
3. **Hot keys / big keys** - `redis-cli --hotkeys` (with LFU policy) and `--bigkeys`; one session key or counter hammered by millions.
4. **Resources** - CPU of the main thread near 100%, memory near `maxmemory` causing evictions, network bandwidth limit of the node, connection count spikes (new connections are expensive - use pooling).
5. **Persistence and replication** - RDB fork / AOF rewrite pauses, replica full resync.
6. **Client side** - connection pool too small, timeouts and retries causing retry storms.

Fixes: replace `KEYS` with `SCAN`, split big keys, cache hot keys locally in the app for a second, use pipelining, spread keys with Redis Cluster/more shards, read replicas for read-heavy data, bigger node or more network bandwidth, schedule persistence off the replicas, use `UNLINK` instead of `DEL`.

Example:

```
redis-cli -h $HOST --latency-history
redis-cli -h $HOST SLOWLOG GET 20
redis-cli -h $HOST --bigkeys
redis-cli -h $HOST INFO stats | grep -E "evicted_keys|instantaneous_ops"
redis-cli -h $HOST INFO clients | grep connected_clients
```

> ***What to say to the interviewer:*** *Because Redis is single-threaded, I look for anything blocking that thread: slow commands in SLOWLOG, big keys, hot keys, fork pauses from persistence, evictions near maxmemory, and network or connection limits. Fixes are replacing KEYS with SCAN, splitting big keys, local caching of hot keys, pipelining and connection pooling, and sharding with Redis Cluster or read replicas.*

Q32. After an outage, how would you conduct a proper post-incident analysis (RCA / blameless postmortem)?

Also asked as: How do you write RCA documentation? Postmortems.

> **In simple words:** After a plane incident, investigators do not just blame the pilot; they look at the whole system - checklists, training, instruments - so the same thing never happens again to any pilot.

Detailed answer

1. **Blameless** - focus on how the system allowed the mistake, not who made it. People share more honestly.
2. **Within 2-5 days**, while memory is fresh; include everyone involved.
3. **Timeline** - detection, escalation, mitigation, resolution, with timestamps from chat, alerts and deploy logs.
4. **Impact** - duration, users/regions affected, SLO budget consumed, revenue/SLA effect.
5. **Root cause and contributing factors** - use "5 Whys" or a fishbone; there is usually more than one cause (a bug + a missing alert + a slow rollback). Verify each cause with evidence, do not assume.
6. **What went well / what went badly / where we got lucky.**
7. **Action items** - specific, owned, dated, tracked in Jira: prevent (fix the bug, add test), detect (new alert), mitigate faster (runbook, automated rollback).
8. **Share and follow up** - publish, review action items until closed.

Example: Structure:

```
Summary | Impact | Timeline | Root cause | Contributing factors
Detection (how/when did we find out?) | What went well | What went wrong
Action items: [ID] [Action] [Owner] [Due date] [Type: prevent/detect/mitigate]
```

> ***What to say to the interviewer:*** *I run a blameless postmortem within a few days, build an evidence-based timeline, quantify the impact, and use 5 Whys to find the root cause and contributing factors instead of blaming a person. The key output is a small set of owned, dated action items for*

prevention, detection and faster mitigation, tracked until they are closed.